

# Flow Past a Circular Cylinder: Reynolds Number Sweep

---

OpenFOAM v2012 · pimpleFoam / pimpleFoam + kOmegaSST · 4-Core Parallel · snappyHexMesh

A parametric study of 2D flow over a circular cylinder (diameter = 1 m) across a sweep of Reynolds numbers, from creeping Stokes flow through laminar vortex shedding onset into the turbulent regime. Developed as part of an OpenFOAM CFD portfolio demonstrating transient flow simulation and the diagnostic process behind getting a solver to reproduce known physics.

---

## Key Results

---

| Case       | Re    | Inlet U (m/s) | Model     | Cd      | Cl amplitude | Regime                                       |
|------------|-------|---------------|-----------|---------|--------------|----------------------------------------------|
| Re_3       | 3     | 4.5e-05       | laminar   | 5.92    | ~0           | Steady, unseparated creeping flow            |
| Re_40      | 40    | 6.0e-04       | laminar   | 1.64    | ~0           | Steady, fixed symmetric recirculation bubble |
| Re_100     | 100   | 1.5e-03       | laminar   | 1.35    | 0.31         | Periodic von Karman vortex shedding          |
| Re_30000   | 3.0e4 | 0.45          | kOmegaSST | 1.21    | 1.09         | Turbulent subcritical shedding               |
| Re_800000  | 8.0e5 | 12            | kOmegaSST | pending | pending      | Supercritical turbulent (run in progress)    |
| Re_5000000 | 5.0e6 | 75            | kOmegaSST | pending | pending      | Transcritical turbulent (queued)             |

Cd is the final or near-converged drag coefficient, Cl amplitude is the peak-to-peak lift coefficient swing divided by two, taken over the last 20% of each run. Re\_800000 and Re\_5000000 use the same mesh and solver recipe but need a much smaller Courant-limited timestep at their higher inlet velocity, so they take on the order of a day each to solve and are not yet finished.

---

## Domain and mesh

---

All cases share the same geometry and mesh recipe:

- Domain: a `[-10, 20] x [-10, 10] x [0, 1]` m box, cylinder of radius 0.5 m centred at the origin.
- `blockMesh` generates a uniform background hex mesh.
- `snappyHexMesh` cuts the cylinder out of the background mesh using a built-in `searchableCylinder` primitive (no STL needed), with surface and distance-based refinement near the wall and in the wake.
- `inlet/outlet` are patch, `front/back` are empty (2D case), `cylinder` is wall. `top/bottom` are patch with a `slip` field condition, not `symmetryPlane` (see lessons below).
- Around 115,000 cells per case.

`snappyHexMesh` is run in parallel across 4 cores (`decomposePar` then `mpirun -np 4 snappyHexMesh -overwrite -parallel`, followed by `reconstructParMesh`), which cuts meshing time from roughly 30 minutes to under two minutes per case versus a serial run, with identical mesh quality.

This mesh approach was chosen after an earlier hand-built O-grid `blockMeshDict` (multiple blocks stitched directly with explicit vertices) produced a topologically disconnected mesh, `checkMesh` reported three separate, unconnected cell regions from non-conformal seams between the inner O-grid blocks and the far-field blocks. Rebuilding around `snappyHexMesh` removed that whole class of bug.

---

## Solver setup

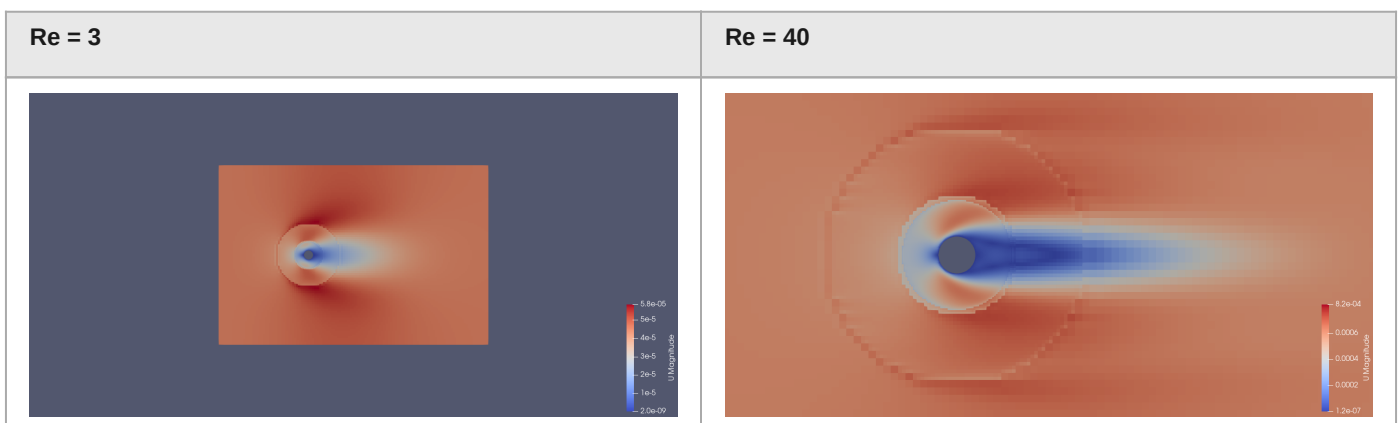
---

- **Solver:** `pimpleFoam`, transient PIMPLE algorithm, no under-relaxation.
  - **Turbulence:** laminar for `Re_3/Re_40/Re_100`, `kOmeGasST` RAS model for the higher-Re cases.
  - **Viscosity:** fixed at  $\nu = 1.5 \times 10^{-5} \text{ m}^2/\text{s}$  for every case, with inlet velocity scaled to hit each target Reynolds number through  $\text{Re} = U \cdot D / \nu$ .
  - **Force monitoring:** `forceCoeffs` function object on the `cylinder` patch, sampled every 10 timesteps.
  - **Time step:** adjustable, Courant-limited (`maxCo 0.8`), with a per-case `maxDeltaT` ceiling tuned to the case's own convective timescale.
- 

## Results

---

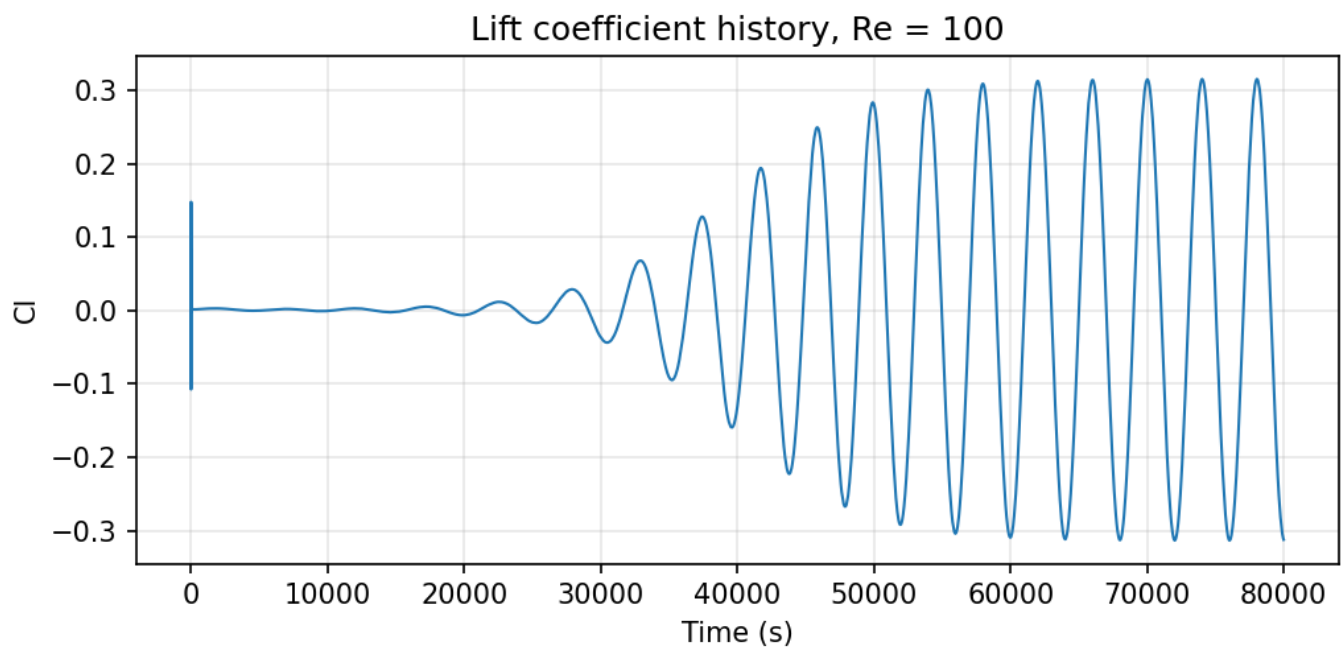
### Velocity contours



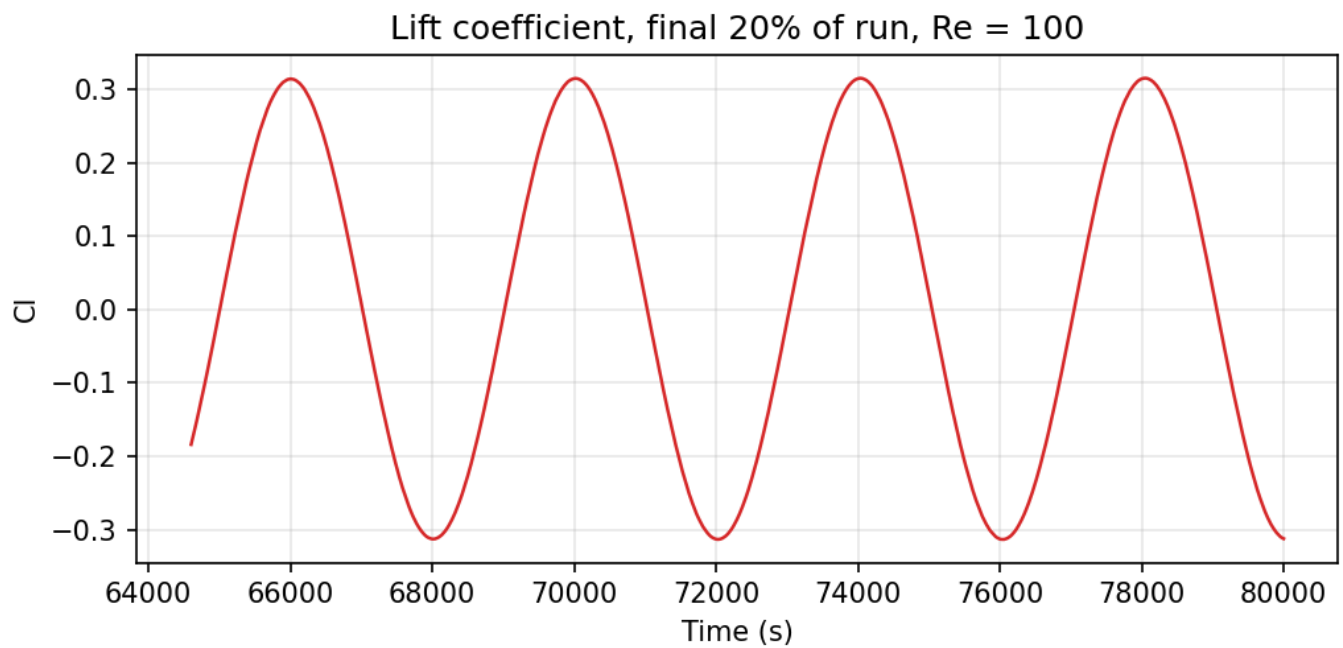


At  $Re = 3$  and  $Re = 40$  the wake stays attached and symmetric about the centreline, matching the unseparated and fixed-bubble regimes expected at these Reynolds numbers. At  $Re = 100$  and above, the wake breaks symmetry and sheds alternating vortices, the signature von Karman street.

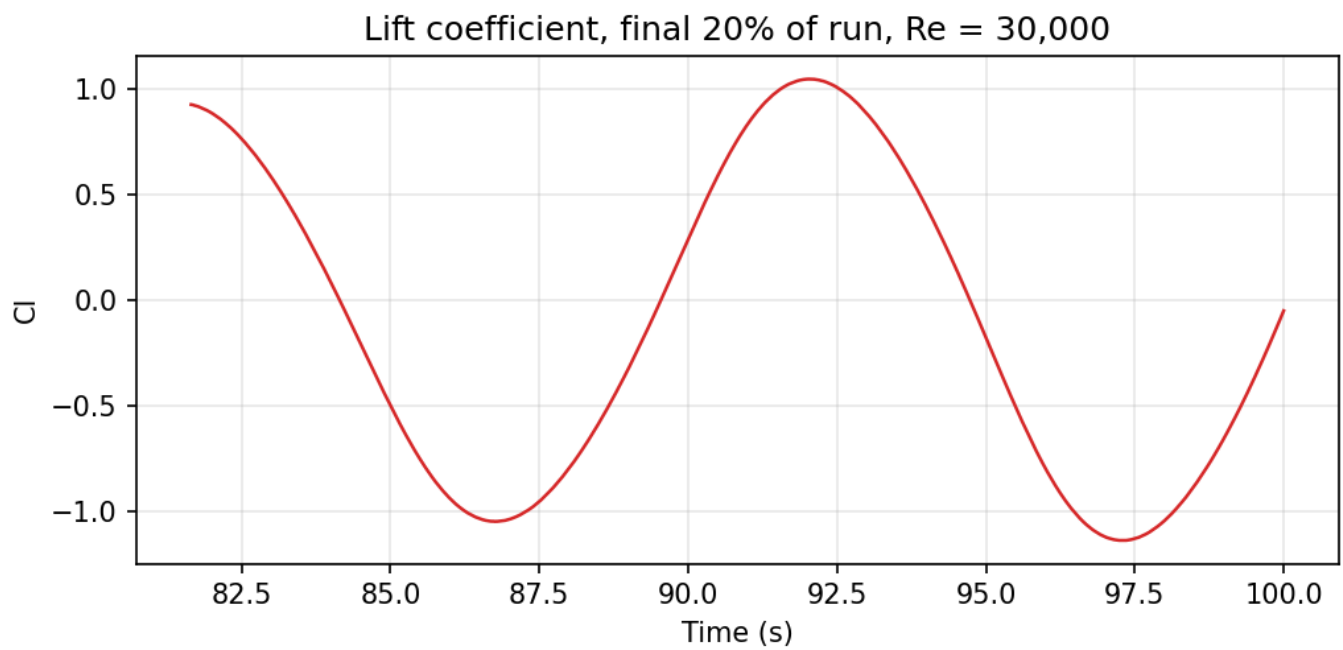
### Lift coefficient history



$Re = 100$  lift coefficient over the full run.  $Cl$  starts near zero, grows as the shedding instability develops, then settles into a clean periodic oscillation.

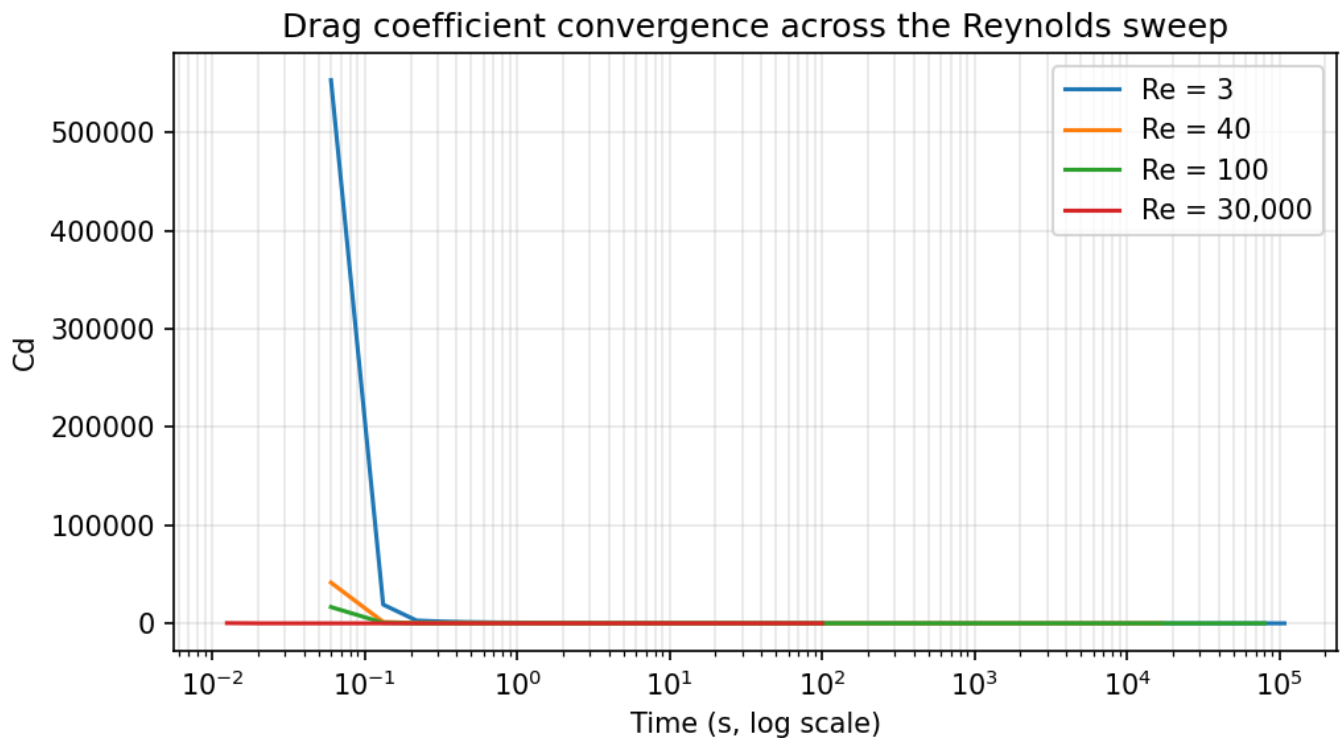


Final 20% of the  $Re = 100$  run, showing the fully developed shedding cycle, period around 4000 s, amplitude about 0.31.



$Re = 30,000$  lift coefficient over the last 20% of its run. Shedding is faster and stronger in this turbulent case, the amplitude is still growing toward saturation at the current end time.

### Drag coefficient convergence



Drag coefficient versus time (log scale) for all four completed cases.  $Re_3$  converges the slowest, a known feature of low-Reynolds-number cylinder drag, while the higher-Re cases settle within their much shorter convective timescales.

---

## Workflow

```
blockMesh (background hex mesh) | decomposePar (background mesh only) | mpirun -np 4 snappyHexMesh  
-overwrite -parallel | reconstructParMesh -constant -mergeTol 1e-6 | checkMesh | rm -rf processor* &&  
decomposePar (re-decompose 0/ fields against final mesh) | mpirun -np 4 pimpleFoam -parallel |  
reconstructPar -> ParaView post-processing
```

The second `decomposePar` pass is required because the cylinder patch only exists in the mesh after parallel `snappyHexMesh` finishes, decomposing the fields before that point leaves the per-processor field files without a `cylinder` boundary entry, which crashes `pimpleFoam` with a `Cannot find patchField entry for cylinder` error.

---

## Debugging lessons

This case went through three rounds of fixes before it produced believable physics, all three were necessary, none alone was sufficient:

**`symmetryPlane` top/bottom boundaries silently forced a symmetric solution.** The original mesh and field boundary conditions used `symmetryPlane` on the top and bottom domain walls. That condition mathematically enforces a mirror-symmetric flow field, which makes the asymmetric von Karman shedding mode impossible no matter how long the case runs. Fixed by changing the patch type to `patch` in `blockMeshDict` and the field condition to `slip` in `0/U`, `0/p`, `0/k`, `0/omega`, `0/nut`.

**Leftover steady-state relaxation factors damped the transient solution.** `fvSolution` carried a `relaxationFactors` block (`p 0.3, U/k/omega 0.7`) left over from an earlier steady-state attempt. Applying SIMPLE-style under-relaxation to a transient PIMPLE run artificially damps the time-accurate update each step and suppresses real unsteady dynamics. Removed entirely.

**endTime was far shorter than the flow's own timescale.** Inlet velocity is scaled down to hit low Reynolds numbers, which makes the convective timescale  $D/U$  and shedding period  $\tau \sim D/(St \cdot U)$  very large in absolute seconds. The original `endTime = 200 s` was less than 1/20th of one shedding cycle at  $Re = 100$ . Increased `endTime` (and the `maxDeltaT` ceiling, to keep wall-clock cost down) per case based on its own convective and shedding timescale.

**A coarse maxDeltaT produced timestep-locked numerical ringing, not real oscillation.** After the fixes above,  $Re = 3$  still showed an oscillating `Cl`, but at constant amplitude with a period of exactly five timesteps, regardless of the (well within bounds) Courant number. That is temporal under-resolution producing numerical ringing around the true steady solution, not physical shedding, which should not exist at  $Re = 3$ . Lowering `maxDeltaT` from 50 s to 5 s removed the ringing and let `Cl` settle to its expected near-zero steady value.

The takeaway: a clean-looking, bounded oscillation in a force coefficient history is not on its own proof of physical vortex shedding, it can just as easily be a numerical artifact, and only checking it against the expected flow regime (steady vs unsteady, oscillation period vs timestep) catches the difference.

---

## Running a case yourself

---

Each `Re_*` folder is a self-contained OpenFOAM case with everything needed to regenerate the mesh and resolve the flow from scratch. Only the generated mesh (`constant/polyMesh`), the parallel decomposition (`processor*`), and intermediate time directories have been omitted to keep the repo lightweight, the final time directory is kept as a sample result.

```
bash cd "Re_100" ./Allrun
```

Turbulent cases (`Re_300000`, `Re_800000`, `Re_5000000`) use `kOmegaSST` and require a `wallDist` entry in `system/fvSchemes` (included), a common gotcha when adapting `fvSchemes` from a laminar case.

---

## Limitations

---

- `Re_800000` and `Re_5000000` are still solving at the time of writing, results will be added once they finish.
- 2D domain with empty front/back patches, no spanwise three-dimensional shedding effects are captured.
- Fixed kinematic viscosity with inlet velocity scaled per case, rather than holding velocity fixed and varying viscosity, this keeps the geometry and mesh identical across the sweep but means each case has a very different physical timescale.
- $Re = 3$  drag coefficient is still drifting by a fraction of a percent per 2000 s of simulated time at the point captured here, a known feature of how slowly low-Reynolds-number cylinder drag approaches its asymptote (related to the Oseen/Stokes paradox).

---

## Software Stack

---

| Tool                | Version | Purpose                               |
|---------------------|---------|---------------------------------------|
| OpenFOAM            | v2012   | CFD solver                            |
| ParaView            | 5.11    | Post-processing and contour rendering |
| Python / matplotlib | 3.x     | Force coefficient plots               |
| Ubuntu (WSL2)       | 24.04   | OS                                    |
| OpenMPI             | 4.0.3   | Parallel communication                |

---

## Repository

---

Part of the [OpenFOAM Portfolio](#), a collection of CFD simulations demonstrating thermal and fluid simulation skills relevant to thermal engineering roles.